1 Introduction

Branch Predictor Implementation

Ali Alfageeh

GitHub repository: https://github.com/fqeh/CompArchAssignment1

Computer Architecture Programming Assignment
25 October 2026

In this assignment, I completed the Pentium M-style hybrid direction predictor in the supplied CBP2 simulator. I replaced the always-taken `pm_predictor` skeleton with a bimodal predictor and a tagged, two-way global predictor. I then tested the state transitions and evaluated all 20 supplied program traces.

What I added

I implemented the predictor tables, tag matching, prediction selection, saturating counters, allocation, least-recently-used replacement, and history updates. I also added repeatable tests, a portable benchmark runner, exact result files, and the before-and-after code snapshots shown in this report.

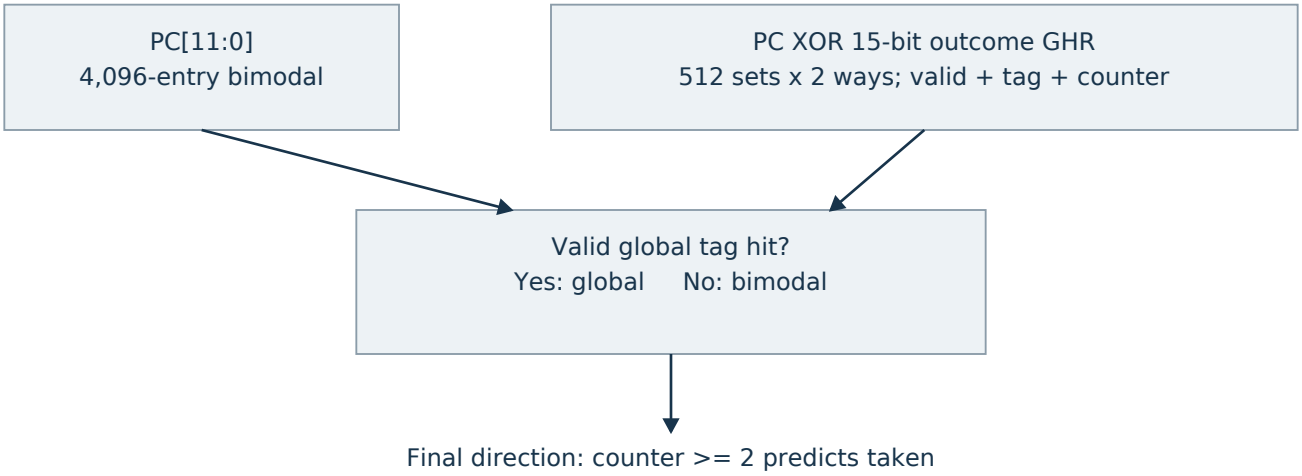
Result	Measured outcome
Completed trace evaluation	80 runs: 20 traces x 4 predictors
Hybrid direction accuracy	94.7291% of conditional branches
Hybrid MPKI	7.259603
Change versus bimodal	29.51% fewer direction misses; better on 19 of 20 traces
Verification	16 predictor tests, 9 driver tests, 40 baseline comparisons

Scope and limitations

I implemented the required direction-prediction portion using the permitted two global ways. I did not implement the loop predictor, branch targets, PIR, or the original PIR hash because they are outside the required scope. My model uses actual conditional-outcome history and a simple XOR. I do not simulate pipeline timing or speculative recovery.

2 Predictor design and assumptions

I used the table dimensions in the assignment diagram and the permitted two-way simplification. I count a global hit only when a valid entry has the requested tag; a matching row by itself is not a hit.



Component	Implemented configuration
Bimodal table	4,096 counters; PC[11:0]; counters start at 1
Global table	512 sets x 2 ways; each entry has valid, 6-bit tag, 2-bit counter
Global history	15 bits; initialized to 0; only conditional outcomes are shifted in
Address/history mapping	H = (PC XOR GHR) & 0x7fff; set = H >> 6; tag = H & 0x3f
Prediction rule	Global counter on a valid hit; bimodal counter otherwise
Miss allocation	After resolution, prefer an invalid way; otherwise replace LRU
Training	Train bimodal on every conditional branch; update or allocate global
Counter threshold	0 or 1 predicts NT; 2 or 3 predicts T

Choices I made: I initialize all bimodal counters weakly not taken and new global entries weakly in the observed direction. I allocate on every global miss and train both components. I document these choices to make the experiment reproducible because the handout does not specify a complete production allocation policy.

My update order: I predict using the old GHR, retain the indices and hit way, resolve the branch, train the tables, update LRU, and finally shift the actual outcome into the GHR. I use the CBP2 driver's immediate resolution of each prediction.

3 Code changes: saved state and storage

The starter update object stored only one index and the Pentium M class had no predictor state. I added the information needed to train the same entries that were used during prediction. I also added private storage for both predictors and their replacement state.

Initial Version

Initial Version | baseline/src/my_predictor.h | lines 57-64

```
57  class pm_update : public branch_update {
58  public:
59          unsigned int index;
60  };
61
62  class pm_predictor : public branch_predictor {
63  public:
64          pm_update u;
```

My Version: prediction record

My Version | src/my_predictor.h | lines 66-73

```
66  class pm_update : public branch_update {
67  public:
68      unsigned int index = 0;       // Bimodal index saved at prediction time.
69      unsigned int set = 0;
70      unsigned int tag = 0;
71      int hit_way = -1;
72      bool conditional = false;
73  };
```

My Version: private predictor state

My Version | src/my_predictor.h | lines 100-105

```
100     pm_update u;
101     unsigned int ghr = 0;
102     unsigned char bimodal[BIMODAL_SIZE];
103     entry global[GLOBAL_SETS][WAYS];
104     unsigned char lru[GLOBAL_SETS] = {};
105     statistics stats;
```

Figure 1. I store the bimodal index, global set, tag, hit way, and conditional flag in `pm_update`. I use `lru[set]` to record the replacement way when both ways are valid.

I keep the prediction-time indices so that the later GHR update cannot redirect training. The driver has one unresolved prediction at a time.

3.1 Initialization and two-bit counters

The original constructor did nothing. I initialize the bimodal counters to 1, the GHR to 0, all valid bits to false, and the initial replacement way to 0. Invalid entries never supply a prediction.

Initial Version

Initial Version | baseline/src/my_predictor.h | lines 66-67

```
66          pm_predictor (void) {
67          }
```

My Version

My Version | src/my_predictor.h | lines 116-119

```
116     pm_predictor() {
117         // Weakly not taken; global entries start invalid; GHR starts at zero.
118         for (unsigned int i = 0; i < BIMODAL_SIZE; ++i) bimodal[i] = 1;
119     }
```

My Version | src/my_predictor.h | lines 87-91

```
87      struct entry {
88          bool valid = false;
89          unsigned char tag = 0;
90          unsigned char counter = 1;
91      };
```

My Version | src/my_predictor.h | lines 107-113

```
107     static void train(unsigned char &counter, bool taken) {
108         if (taken) {
109             if (counter < 3) ++counter;
110         } else {
111             if (counter > 0) --counter;
112         }
113     }
```

Current counter	Prediction	After NT	After T
0: strongly not taken	NT	0	1
1: weakly not taken	NT	0	2
2: weakly taken	T	1	3
3: strongly taken	T	2	3

Figure 2. I clamp each counter at 0 and 3. I initialize a new global tag to 1 or 2 from its first actual outcome so it does not inherit the old tag's confidence.

3.2 Prediction and tag-hit selection

The starter predicted taken for every branch. I now use the bimodal prediction as the fallback, then override it only when a valid global entry matches the computed tag. A nonconditional branch still predicts taken.

Initial Version

Initial Version | baseline/src/my_predictor.h | lines 69-78

```
69          branch_update *predict (branch_info & b) {
70
71              // predict branch outcome
72              u.direction_prediction (true);
73
74              // predict branch target address
75              u.target_prediction (0);
76
77              return &u;
78          }
```

My Version

My Version | src/my_predictor.h | lines 121-142

```
121     branch_update *predict(branch_info &b) override {
122         u = pm_update{};
123         u.conditional = (b.br_flags & BR_CONDITIONAL) != 0;
124         u.direction_prediction(true);
125         u.target_prediction(0);  // Target prediction is outside this assignment.
126         if (!u.conditional) return &u;
127
128         u.index = b.address & (BIMODAL_SIZE - 1);
129         const unsigned int hash = (b.address ^ ghr) & HISTORY_MASK;
130         u.set = hash >> GLOBAL_TAG_BITS;  // HASH[14:6]
131         u.tag = hash & TAG_MASK;         // HASH[5:0]
132         u.direction_prediction(bimodal[u.index] >= 2);
133         for (unsigned int way = 0; way < WAYS; ++way) {
134             const entry &e = global[u.set][way];
135             if (e.valid && e.tag == u.tag) {
136                 u.hit_way = static_cast<int>(way);
137                 u.direction_prediction(e.counter >= 2);
138                 break;
139             }
140         }
141         return &u;
142     }
```

Figure 3. I use PC[11:0] for bimodal, HASH[14:6] for the global set, and HASH[5:0] for the tag. I search both ways and override bimodal only on a valid tag match.

3.3 Training, allocation, and LRU

The original update function was empty. I train the selected entries only after the actual outcome is known. On a global miss, I allocate an unused way first and evict the least recently used way only when the set is full.

Initial Version

Initial Version | baseline/src/my_predictor.h | lines 80-82

```
80          void update (branch_update *u, bool taken, unsigned int target) {
81
82          }
```

My Version

My Version | src/my_predictor.h | lines 144-172

```
144     void update(branch_update *update, bool taken, unsigned int) override {
145         const pm_update &saved = *static_cast<pm_update *>(update);
146         if (!saved.conditional) return;
147
148         // Train using the indices from BEFORE this branch changes the GHR.
149         train(bimodal[saved.index], taken);
150         unsigned int way;
151         if (saved.hit_way >= 0) {
152             ++stats.hits;
153             way = static_cast<unsigned int>(saved.hit_way);
154             train(global[saved.set][way].counter, taken);
155         } else {
156             ++stats.misses;
157             // Prefer unused ways; otherwise evict the least recently used way.
158             for (way = 0; way < WAYS; ++way)
159                 if (!global[saved.set][way].valid) break;
160             if (way == WAYS) {
161                 way = lru[saved.set];
162                 ++stats.replacements;
163             }
164             entry &e = global[saved.set][way];
165             e.valid = true;
166             e.tag = static_cast<unsigned char>(saved.tag);
167             // A new tag must not inherit the evicted branch's confidence.
168             e.counter = taken ? 2 : 1;
169         }
170         lru[saved.set] = static_cast<unsigned char>(1u - way);
171         ghr = ((ghr << 1) | static_cast<unsigned int>(taken)) & HISTORY_MASK;
172     }
```

Figure 4. I train an existing counter on a hit and allocate a tag on a miss. I mark the other way LRU, then shift the actual outcome into the masked 15-bit GHR last.

3.4 Driver and measurable output

I retained the original `predict TRACE` interface and its MPKI line. I added predictor selection and JSON output so the report can use exact miss counts and conditional-branch accuracy rather than rounded terminal values.

Initial Version: conditional miss counting

Initial Version | baseline/src/predict.cc | lines 60-62

```
60              // count a direction misprediction
61
62              dmiss += u->direction_prediction () != t->taken;
```

My Version: the complete simulation loop

My Version | src/predict.cc | lines 48-59

```
48      init_trace(argv[1]);
49      unsigned long long branches = 0, conditional = 0, misses = 0;
50      while (trace *t = read_trace()) {
51          ++branches;
52          branch_update *u = p->predict(t->bi);
53          if (t->bi.br_flags & BR_CONDITIONAL) {
54              ++conditional;
55              misses += u->direction_prediction() != t->taken;
56          }
57          p->update(u, t->taken, t->target);
58      }
59      end_trace();
```

My Version: selecting the comparison predictor

My Version | src/predict.cc | lines 41-46

```
41      std::unique_ptr<branch_predictor> p;
42      if (mode == "pm") p.reset(new pm_predictor());
43      else if (mode == "bimodal") p.reset(new bimodal_predictor());
44      else if (mode == "gshare") p.reset(new gshare_predictor());
45      else if (mode == "always_taken") p.reset(new always_taken_predictor());
46      else return usage(argv[0]);
```

Figure 5. I give each configuration the same trace and outcomes. I compare my hybrid with the original always-taken behavior, a matched bimodal-only predictor, and the supplied gshare example. I exclude target misses because target prediction is outside the required scope.

3.5 A repeatable benchmark runner

The supplied runner depended on `csh` and `dc` and reported only rounded MPKI. I replaced it with a small portable shell entry point and a Python runner. The new runner evaluates all traces with all four predictors and saves exact CSV and JSON results.

Initial Version: beginning of the supplied runner

Initial Version | baseline/run | lines 1-16

```
#!/bin/csh
if ( $1 == "" ) then
    printf "Usage: $0 <trace-file-directory>\n"
    exit 1
endif
if ( ! { cd src; make -q } ) then
    printf "predict program is not up to date.\n"
endif
if ( ! -e src/predict ) then
    printf "predict program is not built.\n"
    exit 1
endif
set trace_list = `find $1 -name '*.trace.*' | sort`
set sum = 0
set n = 0
foreach i ( $trace_list )
```

My Version: portable entry point

My Version | run | lines 1-5

```
#!/bin/sh
# Portable replacement for the supplied csh runner. Author: Ali Alfageeh.
set -eu
root=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
python3 "$root/scripts/benchmark.py" --traces "${1:-$root/traces}"
```

My Version: one measured run

My Version | scripts/benchmark.py | lines 26-35

```
    def run(item):
        path,mode=item
        cmd=[str(ROOT/'src/predict'),str(path),'--predictor',mode,'--json']
        r=subprocess.run(cmd,text=True,capture_output=True,check=True)
        data=json.loads(r.stdout)
        data={'benchmark':path.parent.name,**data}
        assert data['direction_misses'] <= data['conditional_branches'] <= data['branches']
        if mode=='pm':
            assert data['global_hits']+data['global_misses']==data['conditional_branches']
        return data
```

Figure 6. I run each cold-start experiment in an independent process and check branch-count equality and global hit/miss accounting.

3.6 Build and trace-input fixes

I updated the build to C++17 and added test targets. I also fixed the trace-file opening path so missing files fail cleanly and paths containing spaces or quotes work. The trace decoding algorithm itself is unchanged.

Initial Version: file-open error handling

Initial Version | baseline/src/trace.cc | lines 428-433

```
428     FILE *f = fopen (fname, "r");
429     if (!f) {
430         perror (fname);
431     }
432     fread (s, 1, 2, f);
433     fclose (f);
```

My Version: fail before reading a missing file

My Version | src/trace.cc | lines 428-439

```
428     FILE *f = fopen (fname, "r");
429     if (!f) {
430         perror (fname);
431         exit (1);
432     }
433     const size_t magic_bytes = fread (s, 1, 2, f);
434     if (magic_bytes < 2 && ferror(f)) {
435         perror(fname);
436         fclose(f);
437         exit(1);
438     }
439     fclose (f);
```

My Version: test and sanitizer targets

My Version | src/Makefile | lines 15-23

```
15  test: predict test_predictor
16      ./test_predictor
17      python3 ../tests/test_cli.py
18
19  sanitize:
20      $(CXX) -std=c++17 -g -O1 -Wall -Wextra -pedantic \
21          -fsanitize=address,undefined -fno-omit-frame-pointer -I. \
22          -o test_predictor_san ../tests/test_predictor.cc
23      ./test_predictor_san
```

Figure 7. I also quote the trace path, use a string command buffer, and pair popen with pclose. I fail the run on decompression errors and record the full diff in changes.patch.

4 Verification before benchmarking

I checked the predictor with deterministic state tests, an independently structured reference model, and end-to-end input tests. The reference model stores each set as an MRU-to-LRU list; it does not copy the implementation's physical ways or next-victim bit.

Check group	Coverage and result
16 predictor tests	Initialization, bit mapping, valid-bit gating, fallback, global priority, both counter limits, saved indices, invalid-way priority, LRU, confidence reset, GHR masking, nonconditional branches, combined flags, statistics, and randomized reference. All passed.
250,000 reference events	Fixed seed 20261025; random addresses mixed with concentrated conflicts; conditional and nonconditional events. Predictions, relevant table state, LRU victim, history and totals agree.
9 driver tests	Known miss counts, original output, raw/gzip/bzip2 parity, missing input, invalid arguments, statistics, repeatability, corrupt compression and empty input. All passed.
40 baseline checks	Actual starter binaries agree with the always-taken and gshare controls on all 20 traces, within the starter's 3-decimal output.
Address / UB sanitizers	All 16 predictor tests passed with AddressSanitizer and UndefinedBehaviorSanitizer.

Snapshot of the actual predictor test output

```
PASS initialization
PASS hash_partition
PASS valid_bit_gate
PASS bimodal_fallback
PASS global_overrides_bimodal
PASS local_saturation
PASS global_saturation
PASS saved_indices
PASS invalid_way_first
PASS lru_after_hit
PASS replacement_resets_confidence
PASS history_mask
PASS unconditional_bypass
PASS combined_flags
PASS accounting_and_reset
PASS randomized_reference_250000_events
16 / 16 tests passed
sizeof(pm_predictor) = 7744 bytes
```

Figure 8. I recorded this test output after rerunning with `ASAN_OPTIONS=detect_leaks=0`. LeakSanitizer initially failed because the container could not inspect its process tree; I kept address and undefined-behavior checks enabled. I include both logs and do not claim leak checking or full decoder sanitizer coverage.

4.1 A small trace, step by step

To make the replacement behavior visible, I created an eight-branch synthetic example. The addresses deliberately map to global set 0. All values below are decimal; T = taken and NT = not taken. The predictor uses the same production configuration as the benchmark runs.

Step	PC	Old GHR	Tag	Hit	Local	Final	Actual	New GHR
1	1	0	1	miss	NT	NT	T	1
2	3	1	2	miss	NT	NT	T	3
3	2	3	1	way 0	NT	T	NT	6
4	5	6	3	miss	NT	NT	NT	12
5	14	12	2	miss	NT	NT	NT	24
6	27	24	3	way 1	NT	NT	T	49
7	53	49	4	miss	NT	NT	NT	98
8	97	98	3	way 1	NT	T	T	197

State after each update

Step	Way 0: tag / counter	Way 1: tag / counter	Next victim
1	1 / 2	invalid	way 1
2	1 / 2	2 / 2	way 0
3	1 / 1	2 / 2	way 1
4	1 / 1	3 / 1	way 0
5	2 / 1	3 / 1	way 1
6	2 / 1	3 / 2	way 0
7	4 / 1	3 / 2	way 1
8	4 / 1	3 / 3	way 0

Step 3: PC 2 XOR old GHR 3 gives hash 1, so set 0 and tag 1. Way 0 holds a valid tag 1 with counter 2. It predicts T even though bimodal[2] is 1 and predicts NT. The actual outcome is NT, so the global counter becomes 1 and the GHR becomes 6.

Step 4: PC 5 XOR GHR 6 gives hash 3. There is no tag 3, so bimodal predicts NT correctly. Way 1 is LRU because step 3 refreshed way 0. I replace tag 2 with tag 3 and initialize its counter to 1.

Step 8: PC 97 XOR GHR 98 gives hash 3 again. Way 1 now has tag 3 and counter 2, so global predicts T correctly. Its counter increases to 3.

Figure 9. I generated these values with `tests/walkthrough.cc` and saved them in `results/walkthrough.csv`. I created this verification example because the handout's reduced Appendix B uses different sizes, address shifting and initial counter patterns.

5 Experiment setup and aggregate results

I ran every supplied trace from start to finish with a fresh predictor for each configuration. I used the defined 100-million-instruction denominator for each trace, giving two billion instructions per predictor across all 20 traces. I counted 275,461,634 conditional branches in total.

Configuration	State / initialization
Original skeleton	Always taken; no adaptive state
Bimodal control	4,096 two-bit counters, initially 1; same indexing and training as hybrid fallback
Supplied gshare	32,768 counters, initially 0; 15-bit history; original algorithm retained
My hybrid	4,096 bimodal counters plus 512 x 2 tagged global entries; 15-bit history

```
MPKI = 1000 x direction_misses / instruction_count
Accuracy (%) = 100 x (conditional_branches - misses) / conditional_branches
```

Predictor	Direction misses	MPKI	Accuracy
always_taken	172,623,375	86.311688	37.3331%
bimodal	20,597,217	10.298609	92.5227%
gshare	12,611,718	6.305859	95.4216%
pm	14,519,207	7.259603	94.7291%

My hybrid reduces direction misses by **29.51%** relative to bimodal alone and by **91.59%** relative to the original stub. The supplied gshare produces fewer misses: my hybrid has **15.12% more**. I do not claim that the hybrid is universally best.

Storage interpretation

I calculate my hybrid's logical state as 4,096 x 2 + 512 x 2 x (1 + 6 + 2) + 512 + 15 = **17,935 bits**, or about 2.19 KiB. I include validity, tags, counters, two-way LRU and history. For comparison, I calculate 65,551 logical bits (about 8.00 KiB) for gshare, including history, and 8,192 bits (1 KiB) for bimodal.

These are ideal bit-packed state budgets, not measured silicon area. My C++ object occupies 7,744 bytes in this build because counters and flags use bytes and the object includes statistics and interface state. The gshare comparison is not storage-normalized and also has a different initial counter value.

5.1 Results for all 20 supplied traces

I compare MPKI using the same 100-million-instruction denominator for every predictor in each row; lower MPKI is better. I report my hybrid's conditional-branch accuracy in the last column.

Trace	Always T	Bimodal	Gshare	Hybrid	Hybrid accuracy
164.gzip	70.025	15.030	12.473	12.617	91.652%
175.vpr	85.312	14.671	13.415	14.337	88.823%
176.gcc	86.071	19.347	11.254	15.298	91.045%
181.mcf	102.497	33.245	15.837	17.680	91.853%
186.crafty	48.651	13.075	5.837	8.464	90.519%
197.parser	78.421	15.417	10.008	10.357	92.701%
201.compress	76.983	9.356	7.831	8.199	93.031%
202.jess	92.343	8.809	1.562	1.750	98.763%
205.raytrace	109.506	3.806	2.756	3.421	97.196%
209.db	91.451	6.480	3.909	4.613	96.503%
213.javac	109.644	3.181	2.267	2.638	97.968%
222.mpegaudio	110.339	3.154	2.188	2.604	98.000%
227.mtrt	112.022	3.706	2.657	3.268	97.388%
228.jack	77.958	7.691	3.033	3.946	96.691%
252.eon	24.226	9.405	1.807	3.139	95.936%
253.perlbmk	66.833	5.955	2.554	4.700	96.528%
254.gap	75.950	9.675	3.926	4.797	96.533%
255.vortex	70.524	1.916	1.222	1.239	98.881%
256.bzip2	178.257	0.112	0.094	0.076	99.969%
300.twolf	59.221	21.942	21.489	22.046	83.170%

Table 1. I show full-trace results to three decimals and preserve exact counts and higher-precision values in the CSV and JSON files. I calculate aggregates from integer counts before rounding.

My hybrid beats bimodal on 19 traces. I observe one exception, **300.twolf**, where hybrid MPKI is 22.046 versus 21.942. Against the supplied gshare, my hybrid wins only on **256.bzip2**: 0.076 versus 0.094 MPKI. I retain these exceptions in the results.

Because all traces have equal instruction counts, the arithmetic mean of their unrounded MPKI values equals the aggregate MPKI. I weight accuracy by the actual number of conditional branches instead of averaging the individual percentages.

5.2 Global-table activity and exact counts

I recorded global hits, misses and replacements to check that the tagged component was active and that its accounting matched the simulated conditional branches. Global hit rate measures which component supplied the prediction; it is not the same as prediction accuracy.

Trace	Conditional branches	Hybrid misses	Global hit %	Replacements
164.gzip	15,114,596	1,261,706	83.06	2,559,842
175.vpr	12,827,221	1,433,739	56.09	5,631,925
176.gcc	17,083,242	1,529,825	54.43	7,784,140
181.mcf	21,700,420	1,767,979	77.42	4,898,871
186.crafty	8,927,032	846,360	56.55	3,878,211
197.parser	14,190,746	1,035,735	80.06	2,828,189
201.compress	11,764,885	819,948	87.68	1,448,165
202.jess	14,154,055	175,020	92.36	1,080,199
205.raytrace	12,201,864	342,149	62.95	4,520,345
209.db	13,191,770	461,313	86.70	1,753,771
213.javac	12,986,593	263,842	60.98	5,065,748
222.mpegaudio	13,021,205	260,376	61.23	5,046,970
227.mtrt	12,512,685	326,838	66.69	4,167,402
228.jack	11,926,271	394,642	75.30	2,945,275
252.eon	7,724,960	313,939	76.86	1,786,604
253.perlbmk	13,537,118	469,974	72.93	3,663,371
254.gap	13,838,041	479,727	87.01	1,796,293
255.vortex	11,073,761	123,933	83.95	1,775,989
256.bzip2	24,586,276	7,602	99.62	91,544
300.twolf	13,098,893	2,204,560	32.68	8,817,693

Across all traces, I recorded **203,900,607** global predictions (74.02%) and **71,561,027** conditional predictions using bimodal. I also recorded **71,540,547** full-set replacements.

Table 2. I verify that hits + misses = 275,461,634 conditional branches and misses - replacements = 20,480 initial allocations, exactly 20 traces x 1,024 entries. I use these equalities to check cold initialization and allocation accounting.

5.3 Visual comparison of adaptive predictors

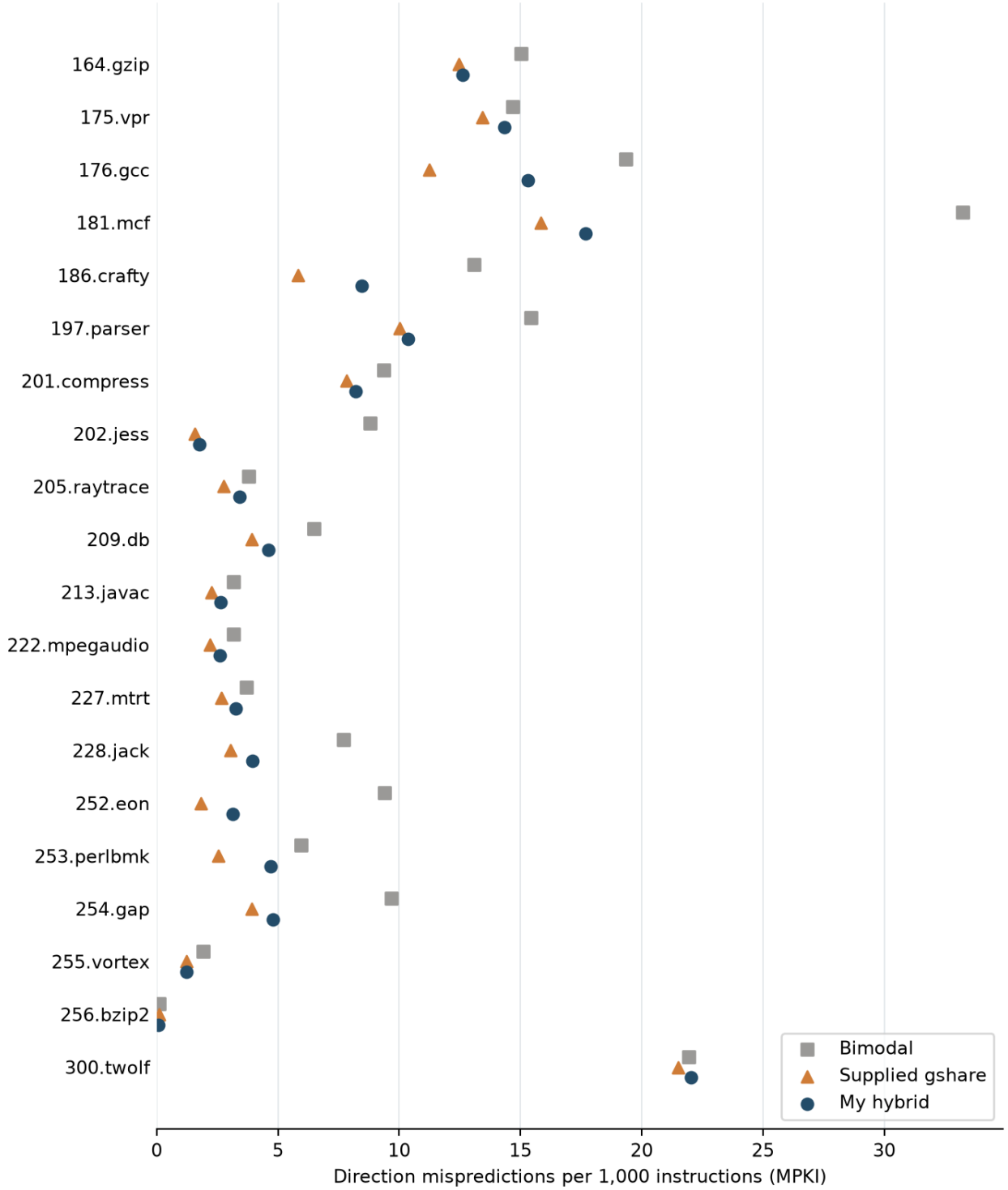


Figure 10. I compare all three adaptive predictors on the same traces. I omit the always-taken stub from this chart to preserve detail and retain its full values in Table 1. My hybrid sits between bimodal and the larger gshare on most workloads.

6 Discussion and limitations

What the results show

I observe that my hybrid adds useful history information beyond the per-address bias captured by bimodal. The strongest reductions include 202.jess (8.809 to 1.750 MPKI), 252.eon (9.405 to 3.139), and 181.mcf (33.245 to 17.680). On 300.twolf, I observe a slightly worse result from the extra component.

I measured an aggregate improvement over bimodal of 3.039 MPKI, or 6,078,010 fewer direction misses over the same two billion instructions. I report my hybrid's 94.7291% conditional accuracy as a direction-prediction result and do not infer an execution-time or energy improvement.

Why gshare can still be better

The supplied gshare has a much larger counter array. My global structure has only 1,024 tagged entries, so different branch/history combinations can replace each other and fall back to bimodal. For example, global hit rate is 32.68% on twolf and 99.62% on bzip2. This is consistent with different amounts of table pressure, but the counters alone do not prove why each benchmark behaves as it does.

Tags reduce some destructive sharing, but a six-bit tag is derived from the same limited 15-bit hash. Different branches and histories can still produce the same hash. The predictor therefore remains an approximation, with aliasing and capacity limits.

What I did not measure

I did not model a reorder buffer, prediction latency, cycle penalties, speculative history, rollback, loop prediction, branch targets, or processor power. I did not perform an equal-storage sweep or tune the design to these traces. The assignment permits the chosen simplifications; they also limit claims about a real Pentium M processor.

Differences from the handout's worked example

I distinguish my full-size configuration from Appendix B's small teaching configuration, which uses four history bits, shifted address bits, and deliberately patterned initial counters. I followed the full-size diagram for dimensions and documented my initialization and replacement policy. I therefore use state tests and a new walkthrough as the numerical oracle, rather than forcing my default results to match the Appendix B tables.

My conclusion

I completed the required hybrid predictor and verified its state updates and measured behavior. The global component improves the matched bimodal control on nearly every supplied workload, while the larger gshare example remains the stronger aggregate predictor in this experiment.

7 Reproducing the work

I published my completed source, starter baseline, tests, raw measurements, and this report in the repository's branch-prediction folder. I excluded the course traces from GitHub. To reproduce the full evaluation, I place those traces under branch-prediction/traces/ and run these commands from the repository root:

```
cd branch-prediction
make -C src
make -C src test
ASAN_OPTIONS=detect_leaks=0 make -C src sanitize
python3 scripts/benchmark.py --jobs 4
python3 tests/test_baseline_equivalence.py
```

One trace with the original output format

```
./src/predict traces/164.gzip/gzip.trace.bz2
```

I recorded `12.617 MPKI` with my completed hybrid, which is the default predictor. I add `--json` for exact counts or `--predictor gshare` to select that control.

Snapshot of the actual benchmark summary

```
Predictor       Misses         MPKI        Accuracy
always_taken    172,623,375   86.311688    37.3331%
bimodal          20,597,217   10.298609    92.5227%
gshare           12,611,718    6.305859    95.4216%
pm               14,519,207    7.259603    94.7291%
80 complete runs; 20 traces; cold predictor for each run.
```

Location	Contents
src/my_predictor.h	Required pm_predictor implementation and retained starter examples
tests/	Unit/reference tests, CLI checks, baseline equivalence and walkthrough
results/	CSV, JSON, test logs, sanitizer logs, walkthrough and provenance hashes
baseline/ and changes.patch	Unchanged starter source plus searchable before/after differences
report/	Final PDF with code snapshots and comparison chart
README.md	Run instructions, configuration and limitations

I recorded the benchmark results with g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0 and Python 3.12.14 on Linux. I require a C++17 compiler, Make, gzip and bzip2 for the simulator; my Python tests and runner use the standard library.

I run the unchanged starter-binary comparison on Linux because its command paths remain unchanged. My current trace reader resolves its utilities through PATH for macOS/Linux portability. I keep the original benchmark measurements and provenance hashes in results/.

8 Requirement check

Assignment requirement	Delivered evidence
Implement pm_predictor	Constructor, predict and update are implemented in src/my_predictor.h.
Bimodal direction prediction	4,096 counters; prediction and saturation tested.
Global direction prediction	512 sets x 2 ways, tags and valid bits; hit selection and training tested.
Global hit has priority	Explicit override in predict; deterministic priority test and walkthrough step 3.
Two ways permitted	Two-way invalid-first allocation and LRU replacement are implemented.
Compile with make	make -C src builds the completed default predictor.
Evaluate supplied traces	All 20 traces and all four comparison modes completed; 80 raw records.
Source, report and README	I published these together in the repository's branch-prediction folder.
Explain changes with snapshots	Sections 3-3.6 show actual starter and completed source excerpts.
Author name	My name and repository link appear on the first page.